

Exact Network Surgery: Functional Invariance and Gradient Plasticity in Reactive Computational Graphs

Abdallah Khemais
ISITCOM, University of Sousse

July 2026

Abstract

Function-preserving network growth techniques such as Net2Net [1] and progressive stacking [5] expand a model’s capacity without destroying its learned function, but existing formulations either tolerate small numerical perturbations or require a full rebuild of the training program, discarding compiled artifacts and complicating optimizer-state management. We formalize *Exact Network Surgery*: the in-place insertion of a residual block into a live computational graph such that (i) the network function is preserved — bit-exactly under explicitly stated floating-point hypotheses — and (ii) the inserted parameters remain trainable immediately after insertion. We prove an identity-morphism theorem for gated residual blocks, a structural-locality theorem showing that a reactive invalidation engine recomputes exactly the downstream cone of the insertion point, and an escape-from-initialization proposition showing that the *Gradient Shadowing* gate α , initialized at zero over a randomly initialized residual branch, receives a generically non-zero gradient at insertion time. We identify and analyze a degenerate configuration — zero-initialized output projections combined with a zero gate — which constitutes an exact saddle point from which gradient descent cannot escape, and explain why our construction avoids it. All three claims are validated empirically on the reference implementation in *NeuroDSL* [13], a reactive define-and-run graph engine in Julia: grafting is bit-exact on every logit tested (0 mismatches out of 1600, preconditions verified rather than assumed); the gate escapes zero at the first optimizer step and unlocks the branch gradients at the second, exactly as predicted; the degenerate configuration exhibits gradients that are identically zero — not merely small — for the entire 600-step run; and surgery cost tracks the downstream cone size with $r = 0.9992$ while the graft-plus-invalidation bookkeeping itself is constant (~ 0.75 ms) across insertion depths.

1 Introduction

Growing a neural network during training — inserting new layers into a model that has already learned — is attractive whenever training a large model from scratch is wasteful: curriculum-style capacity scheduling [5, 6], continual learning, and architecture search all benefit from it. The central difficulty is not conceptual but operational: the insertion must not destroy the learned function, and the surrounding training machinery (optimizer state, compiled kernels, cached activations) must survive the mutation.

Eager frameworks such as PyTorch [11] are define-by-run and therefore support topological mutation natively: one can assign a new module to an attribute and the next forward pass will use it. The cost of surgery in such frameworks is not the mutation itself but its *consequences*: (i) under compilation (`torch.compile`, `jax.jit` [12]), any change to the traced program triggers re-tracing and recompilation of the whole graph; (ii) optimizer state must be re-associated with a mutated parameter set by hand, a common source of silent bugs; and (iii) there is no notion of *partial validity* — the framework cannot distinguish the subgraph whose cached values are still correct from the subgraph invalidated by the mutation.

NeuroDSL [13] takes a different approach: the computational graph is a persistent, reactive DAG. Nodes own their values and their optimizer state; edges carry explicit dependency information; and a mutation triggers an $O(|\text{affected}|)$ invalidation wave rather than a global rebuild. This paper shows that, on top of such an engine, network surgery can be made *exact* — the mutated network is functionally indistinguishable from the original, under hypotheses we make precise — while remaining *plastic*: the new parameters start learning at the very first post-surgery optimization step.

Contributions.

1. A formal definition of grafting on computational graphs and of functional exactness (Section 3).
2. An identity-morphism theorem for gated residual blocks, together with a qualified bit-exactness proposition that states its IEEE-754 hypotheses explicitly (Section 4).
3. A structural-locality theorem: reactive invalidation recomputes exactly the downstream cone of the insertion point, proved by double inclusion over an explicit algorithm (Section 5).
4. *Gradient Shadowing*, a gating scheme in the ReZero family [2], with a proof that the gate escapes its zero initialization in one step, and an analysis of the degenerate double-zero configuration that must be avoided (Section 6).
5. An empirical validation of every claim above (Section 7): bit-exactness with verified preconditions (E1), the three-arm plasticity experiment confirming both the escape and the saddle point (E2), cost-versus-depth measurements confirming locality (E3), and bit-identical training continuation across a real process restart (F3).

2 Related Work

Function-preserving growth. Net2Net [1] introduced function-preserving transformations (*Net2WiderNet*, *Net2DeeperNet*) based on identity-initialized layers; exactness holds for ReLU networks but is perturbed by normalization layers and requires care with weight symmetry (broken in practice by added noise). Progressive stacking [5] and bert2BERT [6] grow Transformers by duplicating layers, accepting a functional perturbation that is repaired by continued training. LEMON [7] restores lossless expansion for LayerNorm-based Transformers. GradMax [8] grows networks by maximizing the gradient norm of the new weights, which is the dual of our concern: we guarantee the new weights have a *usable* gradient path, GradMax optimizes its magnitude.

Zero-gated residuals. Initializing a residual branch so that it initially contributes nothing is a well-studied stabilization device: Fixup [3] zero-initializes the last layer of each residual branch; ReZero [2] multiplies each branch by a learned scalar α initialized at 0 over *randomly initialized* weights; LayerScale [4] uses a learned diagonal gate initialized near zero. Our Gradient Shadowing gate is a ReZero-style scalar; our contribution is not the gate itself but (i) its role in making *surgery* exact rather than training stable, (ii) the explicit saddle-point analysis of the zero-weights-plus-zero-gate configuration, and (iii) its integration with a reactive invalidation engine that makes the surgery local.

Dynamic graph engines. PyTorch [11] is define-by-run; JAX [12] traces pure functions. Neither maintains a persistent reactive graph across steps: validity of cached computation is not a first-class notion. NeuroDSL’s design is closer to incremental computation systems, applied to differentiable programs.

3 Preliminaries

Definition 1 (Computational graph). A computational graph is a tuple $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \text{op}, \theta)$ where $(\mathcal{V}, \mathcal{E})$ is a finite DAG, $\text{op}(v)$ assigns to each non-input node an operator, and $\theta(v)$ its (possibly empty) parameter set. Distinguished subsets $\mathcal{V}_{\text{in}} \subset \mathcal{V}$ (sources) and a node v_{out} (output) induce, by composition along the topological order, a function $F_{\mathcal{G}} : \mathcal{X} \rightarrow \mathcal{Y}$.

Definition 2 (Grafting). Let $e = (u, w) \in \mathcal{E}$ be an edge of $\mathcal{G}$ and $\mathcal{H}$ a computational graph with a single input and a single output, realizing a function $F_{\mathcal{H}}$. The grafting $\mathcal{S}(\mathcal{G}, e, \mathcal{H})$ produces $\mathcal{G}'$ by removing e and adding edges $(u, \text{in}(\mathcal{H}))$ and $(\text{out}(\mathcal{H}), w)$, i.e., the value flowing along e now passes through $\mathcal{H}$.

Defining insertion on an *edge* (rather than "at a node") removes any ambiguity about which consumers of u are rerouted: exactly those along e .

Definition 3 (Functional exactness). $\mathcal{S}(\mathcal{G}, e, \mathcal{H})$ is functionally exact iff $F_{\mathcal{G}'}(x) = F_{\mathcal{G}}(x)$ for all $x \in \mathcal{X}$. It is bit-exact on a set $X_0 \subseteq \mathcal{X}$ of floating-point inputs iff the equality holds at the level of binary representations for all $x \in X_0$ under a fixed evaluation order.

Note that functional exactness over the reals does not imply bit-exactness in floating-point arithmetic, nor conversely; Section 4 treats the two separately.

4 Exact Grafting of Residual Blocks

We instantiate $\mathcal{H}$ as a pre-normalization Transformer block in the Llama family [9]. Its exact sequential structure — which we state faithfully, since the theorem depends on it — is:

$$h = x + \text{Attn}(\text{RMSNorm}(x); \theta_{\text{attn}}), \quad (1)$$

$$H(x) = h + \text{MLP}(\text{RMSNorm}(h); \theta_{\text{mlp}}), \quad (2)$$

where RMSNorm is root-mean-square normalization [10], Attn ends with an output projection W_O , and MLP ends with a down projection W_{down} .

Assumption 1 (No output bias). The projections W_O and W_{down} carry no bias term. (This holds for the Llama architecture [9]; if biases were present, they would additionally have to be initialized to zero.)

Theorem 1 (Identity morphism). Under Assumption 1, if at insertion time t_{ins} the parameters satisfy $W_O = 0$ and $W_{\text{down}} = 0$ (all other parameters arbitrary), then $H = \text{Id}$ over the reals, and consequently $\mathcal{S}(\mathcal{G}, e, \mathcal{H})$ is functionally exact.

Proof. Since Attn terminates in the linear map W_O with no bias, $W_O = 0$ implies $\text{Attn}(\cdot) \equiv 0$ regardless of the attention scores; by (1), $h = x$. Likewise $W_{\text{down}} = 0$ implies $\text{MLP}(\cdot) \equiv 0$, hence $H(x) = h = x$ by (2). Grafting on $e = (u, w)$ replaces the value x_e flowing along e by $H(x_e) = x_e$; every node of $\mathcal{G}'$ therefore computes the same real value as in $\mathcal{G}$, and $F_{\mathcal{G}'} = F_{\mathcal{G}}$. $\square$

Exactness over the reals is not yet bit-exactness: the graft introduces *new floating-point operations* ($x + 0$, and the internal computation of the block). The following proposition states precisely when they are harmless.

Proposition 1 (Qualified bit-exactness). Work in IEEE-754 arithmetic with round-to-nearest. Under the hypotheses of Theorem 1, suppose additionally that on input x :

1. all activations internal to $\mathcal{H}$ upstream of W_O and W_{down} are finite (no $\pm\infty$, no NaN); and
2. no component of the residual stream equals -0.0 .

Then the graft is bit-exact: every downstream node of $\mathcal{G}'$ holds the same binary representation as in $\mathcal{G}$.

Proof. By hypothesis (1), each product inside the zero projections is of the form $0 \times a$ with a finite, which is an exact ± 0.0 , and their sum is ± 0.0 ; hence the branch outputs an exact zero vector. (Hypothesis (1) is necessary: $0 \times \infty = \text{NaN}$, which would propagate.) The only remaining new operations are the residual additions $x_i + 0.0$. In round-to-nearest, $x_i + 0.0 = x_i$ bit-for-bit for every finite $x_i \neq -0.0$; the single exception is $-0.0 + 0.0 = +0.0$, excluded by hypothesis (2). All downstream nodes then receive bit-identical inputs and, computed by the same kernels in the same order, produce bit-identical outputs. $\square$

Remark 1. Without hypothesis (2) the graft remains functionally exact (since $-0.0 = +0.0$ as real numbers) but not bit-exact; without hypothesis (1) it is not even functionally exact in floating point, as a NaN contaminates the residual stream. Empirically, hypothesis (2) is checkable in $O(n)$ before surgery, and hypothesis (1) holds whenever the incoming activations are finite and the randomly initialized internal weights are of moderate scale. The claim of bit-exactness in this paper is always understood under these hypotheses.

5 Topological Stability and Locality

Surgery must leave the cached state of unaffected nodes intact. We formalize the invalidation procedure as Algorithm 1 and prove that it invalidates *exactly* the downstream cone; the NeuroDSL routine `_invalidate_downstream!` implements this algorithm (implementation conformance is an engineering claim, verified by the test suite, not part of the proof).

Algorithm 1 `INVALIDATE( $\mathcal{G}, s$ )` — reactive invalidation from a seed node s

```

1:  $Q \leftarrow [s]$ ; seen  $\leftarrow \{s\}$ 
2: while  $Q$  not empty do
3:    $v \leftarrow \text{pop}(Q)$ 
4:   for each  $w$  with  $(v, w) \in \mathcal{E}$  do
5:     if  $w \notin \text{seen}$  then
6:        $\text{valid}(w) \leftarrow \text{false}$ ; seen  $\leftarrow \text{seen} \cup \{w\}$ ;  $\text{push}(Q, w)$ 
7:     end if
8:   end for
9: end while

```

Definition 4 (Downstream cone). For $s \in \mathcal{V}$, let

$$\mathcal{V}_s^+ = \{v \in \mathcal{V} \mid \text{there is a path of length } \geq 1 \text{ from } s \text{ to } v\}.$$

By convention $s \notin \mathcal{V}_s^+$ unless s lies on a cycle, which the DAG property excludes; in our setting $s = \text{out}(\mathcal{H})$ is the freshly created node, which is born unvalued and needs no invalidation.

Theorem 2 (Structural locality). On a DAG, Algorithm 1 terminates and the set of nodes it marks invalid is exactly $\mathcal{V}_s^+$. Its complexity is $O(|\mathcal{V}_s^+| + |\mathcal{E}_s^+|)$, where $\mathcal{E}_s^+$ is the set of edges internal to the cone.

Proof. Termination. Each node enters seen at most once and is pushed at most once; $\mathcal{V}$ is finite.

Soundness (marked $\Rightarrow$ in cone). We show by induction on the order of marking that every node marked invalid lies in $\mathcal{V}_s^+$. The first nodes marked are the immediate successors of s , which are in $\mathcal{V}_s^+$ via paths of length 1. Inductively, a node w is marked only when popped from the queue along an edge (v, w) with v previously marked or $v = s$; by the induction hypothesis there

is a path $s \rightsquigarrow v$ (possibly empty if $v = s$), which extended by (v, w) gives a path of length ≥ 1 from s to w . Hence $w \in \mathcal{V}_s^+$.

Completeness (in cone $\Rightarrow$ marked). Let $w \in \mathcal{V}_s^+$ and let $s = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k = w$, $k \geq 1$, be a witnessing path. By induction on i : $v_0 = s \in \text{seen}$ and is processed; if v_i is processed, then when it is popped, its successor v_{i+1} is either already in seen (hence was marked and pushed earlier) or is marked and pushed now. Either way v_{i+1} is marked and eventually processed. Thus $v_k = w$ is marked.

Preservation. A node $v \notin \mathcal{V}_s^+$ is never marked by soundness, so $\text{valid}(v)$ is untouched: its cached value, and crucially its optimizer state, survive the surgery.

Complexity. Each node in the cone is popped once and each internal edge scanned once. $\square$

Remark 2 (Consequence for training state). *Theorem 2 is what makes surgery cheap in a reactive engine: parameters outside the cone keep their values and their Adam moments untouched, because state is owned by persistent nodes rather than by an external optimizer object indexed by parameter position. New parameters in $\mathcal{H}$ start with fresh (zero) moments, which is the standard and correct choice: their gradient history is genuinely empty. The interaction with learning-rate schedules is orthogonal and discussed in Section 7.*

6 Gradient Shadowing: Exactness with Plasticity

Theorem 1 buys exactness at a price: with $W_O = W_{\text{down}} = 0$, all parameters *upstream* of the zero projections receive a zero gradient (the chain rule factors through the zero matrices), so the block trains slowly at first — only the zero projections themselves have non-zero gradients. Worse, a natural-looking "belt and braces" combination is fatal:

Proposition 2 (Degenerate double-zero configuration). *Consider the gated block $F_{\mathcal{H}}(x; \alpha) = x + \alpha R(x; \theta)$ with both $\alpha = 0$ and the output projections of R zero-initialized (so $R(x) \equiv 0$). Then at t_{ins} :*

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \left\langle \frac{\partial \mathcal{L}}{\partial F_{\mathcal{H}}}, R(x) \right\rangle = 0 \quad \text{and} \quad \nabla_{\theta} \mathcal{L} = \alpha \cdot \frac{\partial \mathcal{L}}{\partial F_{\mathcal{H}}} \cdot \frac{\partial R}{\partial \theta} = 0.$$

Moreover this configuration is a fixed point of gradient descent: both gradients remain identically zero at every subsequent step, so the block never trains.

Proof. At t_{ins} , $R(x) = 0$ annihilates $\partial \mathcal{L} / \partial \alpha$ and $\alpha = 0$ annihilates $\nabla_{\theta} \mathcal{L}$. Gradient descent therefore leaves $(\alpha, \theta_{\text{out}})$ unchanged, so the same two annihilations hold at the next step; by induction, forever. (Momentum-based optimizers without weight decay share the fixed point, since all moment estimates remain zero.) $\square$

The two exactness mechanisms — zero projections and zero gate — are thus *mutually exclusive*: one must choose exactly one. Gradient Shadowing chooses the gate, following ReZero [2]:

Definition 5 (Gradient Shadowing). *The grafted block is $F_{\mathcal{H}}(x; \alpha) = x + \alpha R(x; \theta)$, where R is a residual branch (any composition of smooth primitives ending in linear projections) with θ drawn from a standard random initialization (so $R(\cdot; \theta) \not\equiv 0$ almost surely), and $\alpha \in \mathbb{R}$ is a learned scalar with $\alpha(t_{\text{ins}}) = 0$.*

Remark 3 (Parallel branch in the reference implementation). *Propositions 2 and 3 are agnostic to R 's internal structure. The reference implementation (`graft_shadow_block!`) instantiates $R(x) = \text{Attn}(N_1(x)) + \text{MLP}(N_2(x))$ as a parallel block (GPT-J style), not the sequential chain of (1)–(2), for an experimental-design reason: in the sequential form, the MLP reads the attention output, so zeroing W_O (needed for the comparison arms of Section 7) starves the MLP branch*

of both signal and gradient — a coupling discovered empirically, where $\nabla_{W_{\text{down}}}$ vanished exactly despite W_{down} never being zero-initialized. The parallel form lets each branch be disabled independently, so the three arms differ only in $(\alpha_0, W_{\text{out}})$ and in nothing else. The sequential form of Theorem 1 remains the one used by the ungated surgery primitive (*insert_block!*).

Exactness at t_{ins} is immediate: $\alpha = 0$ gives $F_{\mathcal{H}} = \text{Id}$, and Proposition 1 applies verbatim with "zero branch output" now produced by the multiplication $\alpha \cdot R(x) = 0 \times R(x)$, requiring the same finiteness hypothesis on $R(x)$. Plasticity is the content of the next proposition.

Proposition 3 (Escape from initialization). *Let $g = \partial\mathcal{L}/\partial F_{\mathcal{H}}$ be the incoming gradient at the graft site on the first post-surgery batch. Then at t_{ins} :*

$$\frac{\partial\mathcal{L}}{\partial\alpha} = \langle g, R(x; \theta) \rangle,$$

which is non-zero for almost every random draw of θ (with respect to any initialization distribution absolutely continuous w.r.t. Lebesgue measure), provided $g \neq 0$. After one SGD step with learning rate η , $\alpha_1 = -\eta \langle g, R(x; \theta) \rangle \neq 0$, and from the second step onward $\nabla_{\theta}\mathcal{L} = \alpha_1 g \partial R/\partial\theta$ is generically non-zero: the entire branch trains.

Proof. The expression for $\partial\mathcal{L}/\partial\alpha$ is the chain rule applied to $F_{\mathcal{H}} = x + \alpha R$. The map $\theta \mapsto \langle g, R(x; \theta) \rangle$ is real-analytic in θ (composition of matrix products, softmax, RMSNorm and smooth activations) and not identically zero — e.g., scaling the output projection scales R linearly, so the function is non-constant along that direction unless $g \perp$ every achievable $R(x; \theta)$, which fails for generic $g \neq 0$ since the achievable set spans the output space under standard architectures. A non-trivial real-analytic function vanishes on a Lebesgue-null set; hence the gradient is non-zero almost surely. The one-step update and the subsequent non-vanishing of $\nabla_{\theta}\mathcal{L}$ follow by substitution. $\square$

Remark 4 (Alternative: Net2Net-style plasticity). *If a scalar gate is undesirable, the symmetric choice is also sound: keep $W_O = W_{\text{down}} = 0$ with no gate (α fixed at 1). Then $\partial\mathcal{L}/\partial W_{\text{down}}$ is proportional to the (generically non-zero) pre-projection activations, so the zero projections move at the first step, unlocking the upstream gradients at the second. This is the Fixup/Net2DeeperNet regime [3, 1]; it trades the extra parameter α for a one-step-slower "warm-up" of the upstream weights. What is not sound is combining the two (Proposition 2).*

Remark 5 (Role of the reactive engine). *Gradient Shadowing requires the graph to hold, at the same time, a forward function that is exactly the identity and a backward rule set that sees through it. In NeuroDSL this is natural: forward dispatch and gradient-rule registration are decoupled, and the surgery installs both atomically inside one invalidation epoch, so no intermediate (inconsistent) graph state is ever observable by a training step.*

7 Experimental Results

Every theorem and proposition in this paper is validated by a dedicated experiment on the reference implementation (NeuroDSL, Julia, Float32, CPU). All protocols fix a master seed *before* any run and apply no post-hoc filtering; per-arm and per-depth seeds are derived deterministically from it. One protocol parameter was recalibrated once: the initial budget (vocabulary 256, 200 steps) produced too weak a learning signal to discriminate the arms of E2; it was changed to vocabulary 50 and 600 steps in a single decision, before analysis, and not revisited afterwards.

7.1 E1: Bit-Exactness (Theorem 1, Proposition 1)

A 4-block Llama-style model ($d = 128$, 4 heads) is evaluated on a held-out batch; a Gradient-Shadowing block is grafted between blocks 2 and 3; the logits are recomputed with no parameter

update and compared at the binary level (`reinterpret(UInt32, ·)`). Crucially, the preconditions of Proposition 1 are *checked*, not assumed: a scan of the residual stream at the graft site found 0/4096 components equal to -0.0 , and 0/4096 non-finite values in $R(x)$. Under these verified preconditions, all 1600 logits are bit-identical (0 mismatches, $\max|\Delta| = 0.0$) — strict binary equality, a stronger statement than the approximate function preservation reported by Net2Net or bert2BERT.

7.2 E2: Plasticity and the Saddle Point (Propositions 2, 3)

Three arms share the same initial model, data stream, and random draw of θ ; they differ *only* in $(\alpha_0, \text{zero-init of } W_{\text{out}})$, made possible by the parallel branch of Remark 3. Training runs 600 SGD steps; Table 1 reports the grafted block’s gradient signals.

Table 1: E2 — three-arm plasticity experiment (600 steps, shared master seed). "First step" columns give the first optimizer step at which the quantity becomes non-zero; "never" means it remained *exactly* zero ($= 0$, no tolerance) for the entire run.

Arm	α_0	W_{out}	$\partial\mathcal{L}/\partial\alpha$ at $t=1$	First step		
				$ \alpha > 0$	$\ \nabla_\theta\ > 0$	$\max\ \nabla_\theta\ $
(a) Grad. Shadowing	0	random	1.64×10^{-2}	1	2	3.28×10^{-2}
(b) Net2Net-style	1	zero	0 ($R \equiv 0$)	—	1	1.65×10^0
(c) Degenerate	0	zero	0	never	never	0 (exact)

Three predictions are confirmed at the resolution of a single optimizer step. *Arm (a)*: $\partial\mathcal{L}/\partial\alpha = 1.64 \times 10^{-2} \neq 0$ at the very first backward (Proposition 3); α moves at step 1; ∇_θ becomes non-zero at step 2, once $\alpha \neq 0$ reopens the chain rule — the exact two-step escape the proposition derives. *Arm (b)* is the mirror image predicted by the Net2Net remark: $\partial\mathcal{L}/\partial\alpha = 0$ at step 1 (since $R(x) \equiv 0$) while $\nabla_\theta \neq 0$ immediately (the zero projections themselves receive gradient), unlocking the upstream weights one step later. *Arm (c)*: over all 600 steps, $\partial\mathcal{L}/\partial\alpha$, $\|\nabla_\theta\|$, and $|\alpha|$ remain *identically* zero — an empirical confirmation of an impossibility result (Proposition 2), at exact-zero resolution rather than within a tolerance. The host network’s own loss still decreases in arm (c) (its pre-existing parameters keep training); the frozen quantity is the grafted block, not the model — precisely the failure mode Proposition 2 warns is silent.

Figure 1 plots the full 600-step trajectories behind Table 1. The loss panel makes the silence of the failure mode visible: the three arms are nearly indistinguishable in loss, yet the gate and gradient panels show arm (c) exactly flat at zero throughout, while arms (a) and (b) each escape at the step the corresponding proposition predicts.

An honest magnitude caveat: in arm (a), $|\alpha|$ remains small throughout the run ($\sim 10^{-3}$, final 2.7×10^{-4}), so while the escape is mathematically confirmed, the block’s *contribution* to the function stays modest at this scale and duration — the concern anticipated in Section 8 about the unbounded-below magnitude of $\partial\mathcal{L}/\partial\alpha$, now observed. Since E2 uses one run per arm (by design: the arms are a controlled comparison, not a distribution estimate), we make no claim about the ordering of final losses across arms.

7.3 E3: Surgery Cost vs. Insertion Depth (Theorem 2)

On an 8-block model (412 graph nodes), a graft is performed after block k for $k = 1, \dots, 7$; each sample uses a fresh, fully-evaluated model, and the cost is split into *graft* (block construction + rewiring + invalidation, no evaluation) and *recompute* (the first demand-driven forward, which re-executes exactly the invalidated cone). Timings are trimmed medians over 15 repetitions; depths are measured in shuffled order within each repetition — a fixed order was found to inflate

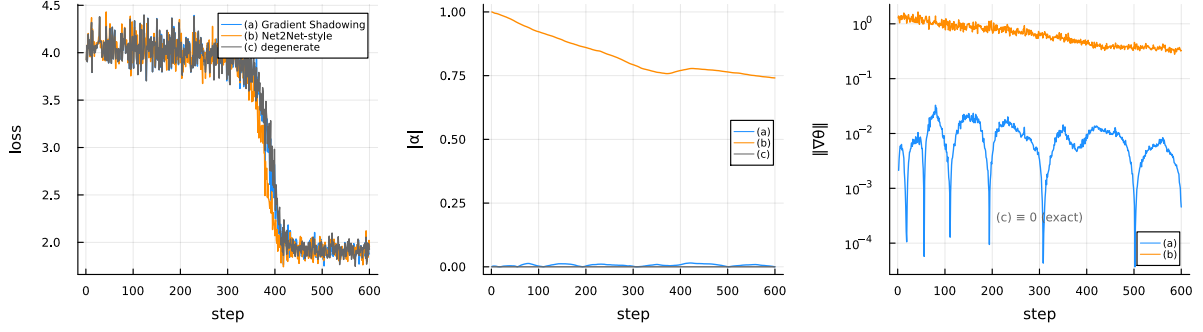


Figure 1: E2 dynamics for the three arms over 600 steps: loss (left), gate magnitude $|\alpha|$ (center), and branch gradient norm $\|\nabla_\theta\|$ on a log scale (right). Arm (c) is exactly zero throughout in both the gate and gradient panels and is annotated rather than plotted, since an exact zero has no representation on a logarithmic axis.

the first-measured depth’s graft time by $\sim 2.6\times$ (a position artifact, removed by the shuffle, not a property of the engine).

Table 2: E3 — surgery cost vs. insertion depth k (8-block model, 412 nodes, trimmed median over 15 shuffled repetitions).

k	Cone $ \mathcal{V}_k^+ $	Graft (ms)	Recompute (ms)	Total (ms)
1	289 (70.1%)	0.745	14.503	15.248
2	248 (60.2%)	0.745	12.626	13.371
3	207 (50.2%)	0.754	11.072	11.826
4	166 (40.3%)	0.745	9.067	9.812
5	125 (30.3%)	0.741	7.660	8.401
6	84 (20.4%)	0.750	5.590	6.340
7	43 (10.4%)	0.751	4.178	4.929

Three observations. First, recomputation cost tracks cone size with $r = 0.9992$ and decreases strictly monotonically — the direct validation of Theorem 2: cost is governed by $|\mathcal{V}_k^+|$, not by the total graph size. Second, the graft-plus-invalidation bookkeeping is *constant* (0.741–0.754 ms) across all depths, independent of cone size — the signature of an $O(1)$ -per-edge consumers index rather than a per-node rule scan; the surgery itself costs less than one fifth of even the smallest recomputation. Third, a linear fit gives $\text{recompute} \approx 0.042 \text{ ms/node} \times |\mathcal{V}_k^+| + 2.24 \text{ ms}$: the non-zero intercept is the $O(|\mathcal{V}|)$ walk of the cached topological order inside the demand-driven evaluator, paid even when only 43 nodes need recomputation. This residual interpreter overhead is exactly what a graph-compilation layer (flat precomputed execution order with $O(1)$ dirty marking) is designed to eliminate, and we leave its removal to that companion line of work.

Figure 2 plots recompute and graft cost against cone size directly: the linear trend and the flat bookkeeping line are both visible at a glance, and the fitted intercept is the 2.24 ms evaluator overhead discussed above.

7.4 F3: Continuity Across a Real Process Restart

Surgery is only useful if the mutated model, including its optimizer state, survives the full lifecycle of real training — interruption included. Three independent Julia *processes* are compared: a control run of 500 steps with a graft at step 300; a run A executing the first 400 steps (graft included) then serializing graph and AdamW state and terminating; and a fresh process B that

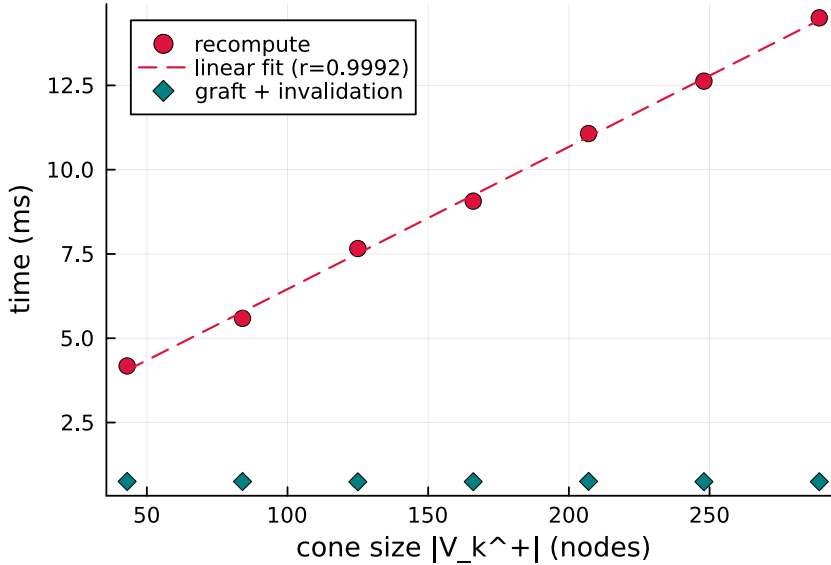


Figure 2: Surgery cost vs. downstream cone size $|V_k^+|$ on the 8-block model of Table 2 ($k = 1, \dots, 7$). Recompute time (circles) grows linearly with cone size ($r = 0.9992$); graft-plus-invalidation bookkeeping (diamonds) is flat at ~ 0.75 ms regardless of cone size.

reloads everything from disk (never re-invoking the model constructor) and runs the remaining 100 steps. The concatenated A+B loss sequence is *bit-identical* to the control on all 500 steps. Falsifiability check: deliberately discarding the loaded AdamW moments in B makes the run diverge from the control at exactly step 402 — the first step at which corrupted optimizer state can affect a weight update — confirming the comparison is sensitive to precisely the state whose preservation is claimed. Figure 3 shows all three loss curves: the control and the correctly-resumed run overlap exactly, while the broken-resume run visibly departs right after the restart marker.

7.5 Beyond Correctness: Utility at Equal Compute

E1–E3 and F3 establish that surgery is exact, local, plastic, and durable; they say nothing about whether growing a model *helps*. A pre-registered companion study on the induction task (commitment to report the outcome recorded before results were seen) found what an honest reading of the growth literature would predict: at equal *step* count, the grafted model slightly trails training-from-scratch (mechanically — it spends fewer steps at the final architecture); at equal *compute* (the axis Net2Net and its successors actually claim, with the 3-vs-4-block per-step cost ratio measured at 0.76 rather than assumed), a paired 20-seed comparison shows one statistically significant window: an early advantage for grafting at 16.7% of the budget (95% CI on the paired loss difference $[+0.16, +0.56]$, excluding zero), which fades to statistical indistinguishability from 33% of the budget onward as both regimes saturate the task. We report this asymmetry as-is: the load-bearing claims of this paper are exactness and locality, not a training-efficiency advantage, and the task’s early saturation limits what any utility comparison can resolve at this scale. Figure 4 shows the full paired-difference curve: the confidence interval clears zero only in the leftmost window.

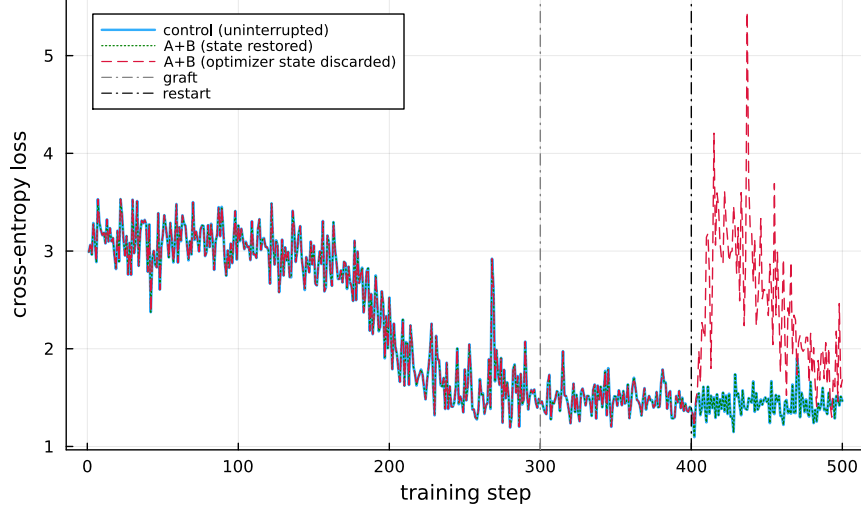


Figure 3: Training loss through a graft (step 300) and a real process restart (step 400). The control run and the correctly-resumed A+B run are bit-identical for all 500 steps; a run that discards the loaded AdamW moments diverges from both at exactly step 402.

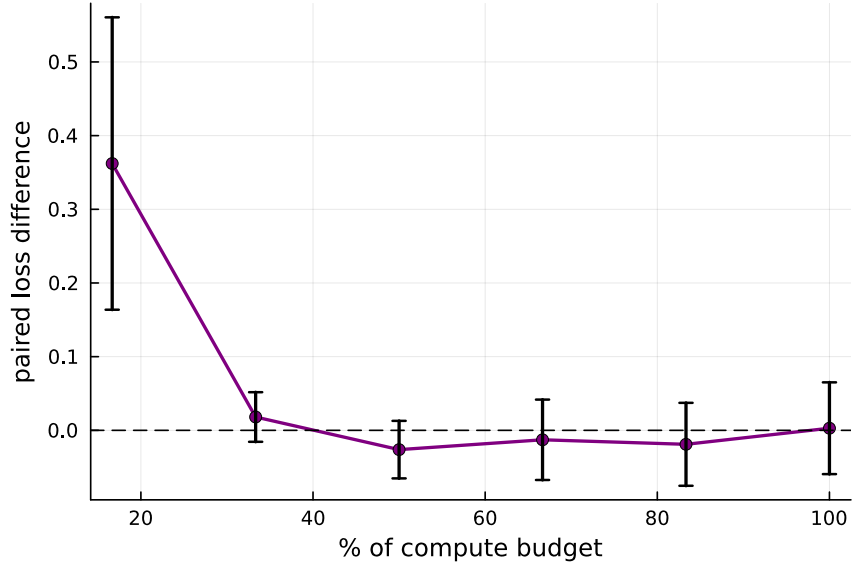


Figure 4: Paired loss difference (loss from scratch – loss grafted) at equal compute, mean $\pm$ 95% CI over 20 seeds (normal approximation). Positive values favor grafting. The interval excludes zero only at 16.7% of the budget.

8 Limitations

Bit-exactness is guaranteed only under the stated IEEE-754 hypotheses and a fixed kernel and evaluation order; frameworks with non-deterministic reductions (e.g., atomics in some CUDA kernels) preserve functional but not bit-level equality, and our measurements are CPU/Float32 — re-verifying E1 on GPU and in bf16 is mechanical but remains to be done. The almost-sure statement of Proposition 3 does not bound the *magnitude* of $\partial\mathcal{L}/\partial\alpha$, and E2 makes this concrete: the escape occurs at step 1 as proved, but $|\alpha|$ stays near 10^{-3} over 600 steps on this task, so the grafted block’s functional contribution ramps up slowly — GradMax-style [8] initialization of θ could be combined with our gate to control this magnitude. E2 uses a single controlled run per arm; its claims are about gradient signals (exact zeros and first-nonzero steps), not about loss distributions. The utility study is confined to a toy task that saturates early. Finally, our locality result concerns validity propagation; memory locality (buffer reuse across the surgery) is engine-specific and out of scope, and the measured ~ 2.2 ms evaluator intercept in E3 bounds how far locality alone can reduce small-cone surgery cost without a compilation layer.

9 Conclusion

Network surgery becomes exact when the graft is an identity morphism, and remains plastic when exactness is carried by a single zero-initialized gate over a randomly initialized branch — never by both mechanisms at once, on pain of an inescapable saddle point. A reactive graph engine makes the operation local: exactly the downstream cone is recomputed, and all other nodes keep their values and optimizer state. Every one of these statements now has a measurement attached to its proof: bit-exact grafting with verified preconditions (0/1600 mismatches), a two-step escape confirmed at the resolution of individual optimizer steps, a saddle point whose gradients stay at exact zero for 600 steps, locality with $r = 0.9992$ against cone size over constant ~ 0.75 ms bookkeeping, and training that resumes bit-identically across a real process restart. Together, these results turn "growing a network mid-training" from a fragile engineering trick into an operation with a specification, a proof, and a passing test suite.

References

- [1] Chen, T., Goodfellow, I., Shlens, J. (2016). Net2Net: Accelerating Learning via Knowledge Transfer. *ICLR*.
- [2] Bachlechner, T., Majumder, B.P., Mao, H., Cottrell, G., McAuley, J. (2021). ReZero is All You Need: Fast Convergence at Large Depth. *UAI*.
- [3] Zhang, H., Dauphin, Y.N., Ma, T. (2019). Fixup Initialization: Residual Learning Without Normalization. *ICLR*.
- [4] Touvron, H., Cord, M., Sablayrolles, A., Synnaeve, G., Jégou, H. (2021). Going Deeper with Image Transformers. *ICCV*.
- [5] Gong, L., He, D., Li, Z., Qin, T., Wang, L., Liu, T.-Y. (2019). Efficient Training of BERT by Progressively Stacking. *ICML*.
- [6] Chen, C., Yin, Y., Shang, L., Jiang, X., Qin, Y., Wang, F., Wang, Z., Chen, X., Liu, Z., Liu, Q. (2022). bert2BERT: Towards Reusable Pretrained Language Models. *ACL*.
- [7] Wang, Y., et al. (2024). LEMON: Lossless Model Expansion. *ICLR*.
- [8] Evci, U., van Merriënboer, B., Unterthiner, T., Vladymyrov, M., Pedregosa, F. (2022). GradMax: Growing Neural Networks using Gradient Information. *ICLR*.

- [9] Touvron, H., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*.
- [10] Zhang, B., Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS*.
- [11] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*.
- [12] Bradbury, J., et al. (2018). JAX: Composable Transformations of Python+NumPy Programs. <http://github.com/google/jax>.
- [13] Khemais, A. (2026). NeuroDSL: A Reactive Computational Graph Framework for Deep Learning in Julia. *Preprint*.